

1 — AttestPay

Agentic spending cards, with provable history

Scoped, revocable payment delegations any AI agent can plug in and pay with — now with

every payment proven cross-chain onto Creditcoin.

`github.com/LSUDOKO/AttestPay`

Track: AI (primary) · DeFi (secondary)



2 — The problem

An AI agent that needs to pay has two bad options:

- **Give it a key.** It can now spend everything, forever.
- **Put a human in the loop.** It is no longer an agent.

AttestPay already solved that part: a card is a scoped, revocable delegation.

But a card builds no reputation. Ask "has this agent paid reliably?" and the only answer is "trust our database". An agent cannot carry its track record anywhere, and nobody else can verify it.



3 — The solution

Every confirmed payment is **proven onto Creditcoin** — no oracle, no bridge.

```
agent pays USDC on Base
  ↓
anchored on an attested chain
  ↓
Attestcoin attestors reach consensus
  ↓
proof verified ON-CHAIN by the Block Prover precompile (0x0FD2)
  ↓
AgentCredit on Creditcoin – readable by any dApp, trusting nobody
```

An agent's payment history becomes portable, public, and checkable.

4 — How it works

Chain	Contract	Role
Base	(existing) ERC-7710 delegation	The money actually moves
Ethereum Sepolia	<code>PaymentAnchor</code>	Records the payment's facts on an attested chain
Creditcoin CC3	<code>AttestPayASC</code>	Proves the anchor, decodes it, keeps the credit record

`spend()` enqueues via one `onChargeConfirmed` hook, so every payment path feeds the pipeline. A background worker drives persisted state:

`pending → anchoring → anchored → attested → proving → verified`

5 — Why the anchor is on Sepolia, not Base

We checked what the protocol actually attests:

```
cast call 0x...0fd3 "get_supported_chains()" --rpc-url <creditcoin>  
→ chainKey 3 → Ethereum mainnet  
chainKey 1 → Ethereum Sepolia
```

Base is not attested. A Base transaction cannot be proven into Creditcoin at all.

So `PaymentAnchor` lives on Sepolia and records the Base payment's facts; that transaction is what gets proven.

What the proof establishes: this anchor record, with exactly these values, was in an attested block. Trustlessly.

What it does not: that the Base payment happened — our server writes the anchor.

Every anchor records the Base tx hash and the anchorer's address, so the claim is

6 — The decision that matters

The obvious signature is unsound:

```
// UNSOUND  
verifyPayment(proof, cardId, amount, from, memo)    // ← facts unchecked
```

The proof and the facts are **independent**. One valid proof of *any* attested transaction would let a caller staple on arbitrary payment data and mint unlimited "verified" credit history.

```
// What we built  
verifyPayment(height, txBytes, merkleProof, continuityProof)
```

The proof **and nothing else**. Every field is decoded out of the proven transaction bytes, so the facts *are* the proof.

7 — Agent credit history

```
struct AgentCredit {  
    uint256 totalPayments;           uint256 totalVolume;  
    uint256 firstPaymentAt;         uint256 lastPaymentAt;  
    uint256 withinTermsPayments;    uint256 termsCheckedPayments;  
}
```

Two honesty decisions, because a credit score invites more trust than data usually supports:

- **termsCheckedPayments** is the denominator, not **totalPayments**. A card with no registered terms does **not** score a free 100% compliance rate.
- **The grade is a published formula**, printed in the tool output: count (≤ 40) + volume (≤ 30) + history days (≤ 30), scaled by the within-terms rate. A summary of

8 — What the agent sees

Four MCP tools, offered only when the integration is configured:

Tool	Answers
<code>verify_payment</code>	Where is this payment in the pipeline?
<code>payment_receipt</code>	Three-chain receipt + what the proof establishes
<code>credit_score</code>	This card's verified history and standing
<code>cross_chain_status</code>	Attestation lag and queue health

Responses are written for a model that will relay them: ISO timestamps, decimal USDC,

an explorer link per leg, and the trust model **verbatim** — so an agent telling a human

"cryptographically verified" can say exactly what that covers

9 — Verified, not asserted

```
$ bun run packages/engine/scripts/attestcoin-probe.ts
```

- ✓ chain 102031 (Creditcoin CC3 Testnet)
- ✓ 2 attested source chains · Base is NOT attested → anchoring required
- ✓ attested 11,688,220 vs source head 11,688,259 → lag 39 blocks (~8 min)
- ✓ prover agrees with the precompile (drift 0)
- ✓ proof generated: 2688 txBytes · 7 Merkle siblings · 1 continuity root

Solidity	42 tests — including the decoder run against real prover output , not our own fixtures
TypeScript	408 tests — the Attestcoin surface tested with the integration both on and off
Docs	Every protocol claim paired with the command that checks it

10 — Team & roadmap

Built this hackathon: the anchor + ASC contracts, the proof pipeline, the credit and terms registries, 4 MCP tools, 6 REST endpoints, the dashboard's Cross-Chain pane, and SigNoz instrumentation of the whole proof lifecycle.

Next:

1. **Base as an attested source chain** closes the one trust gap — the design already decodes from proven receipt logs, so it becomes a change of source chain, not a redesign.
2. **Period-budget verification on-chain**, once the ASC can see the period